

Escuela Politécnica Nacional



Facultad de Ingeniería en Sistemas

Estadística y Programación para Ciencia de Datos

Informe de Práctica N. 2

Clasificación de variables y población vs. Muestra

David Amanta

Christian Suárez

Fecha de entrega

23/10/2026

2026 B

https://github.com/DavidAmanta/IDSD422-lab-muestreo-poblacion.git

1. Introducción

RetailAndina S.A. necesita clasificar las variables de su base de fidelización antes de analizarla y demostrar que una muestra de 20 clientes puede representar a los 400 registros de la base. Como analista de BI, verifiqué la clasificación en código y comparé tres diseños de muestreo contra el parámetro poblacional.

2. Objetivos

- Clasificar las variables de la base de fidelización y comparar tres diseños de muestreo contra el parámetro poblacional.
- Clasificar las 7 variables con el criterio y verificar el tipo en código;
- Declarar el orden de las variables ordinales;
- Comparar conveniencia, aleatorio simple y estratificado contra el parámetro;
- Interpretar el sesgo y la variabilidad e informar sus límites

3. Desarrollo de la práctica

3.1 Fork del repositorio del laboratorio

Para el desarrollo de esta practica como primer paso realice un fork desde Git-Hub al repositorio de Christian Suárez.

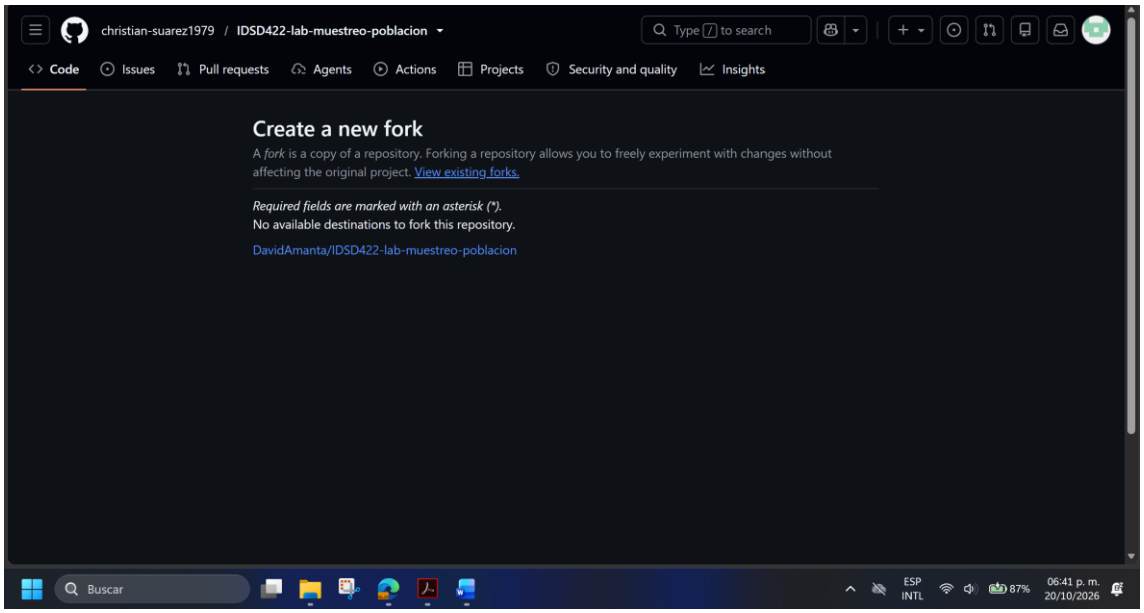


Figura 1. Fork al repositorio original de Christian Suárez.

3.2 Clonar el repositorio en mi máquina

Se clona el repositorio con el comando git clone + el link de mi proyecto, posteriormente se abre desde Visual Studio Code, pero se ejecuta el código desde Google Colab.

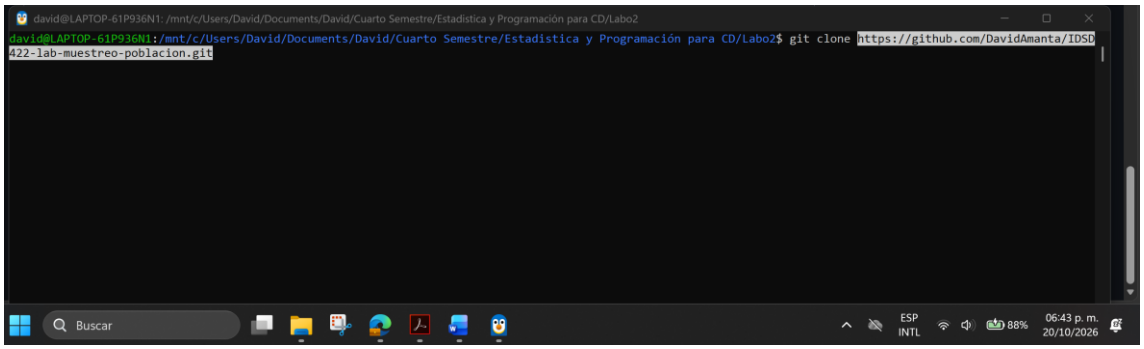


Figura 2. Clonación del fork en mi máquina

Desde Visual Studio Code, se tiene la siguiente estructura, donde procedí a realizar el código tanto para los archivos .py, como para los archivos .R desde Visual Studio Code con las extensiones necesarias para que los Scripts funcionen:

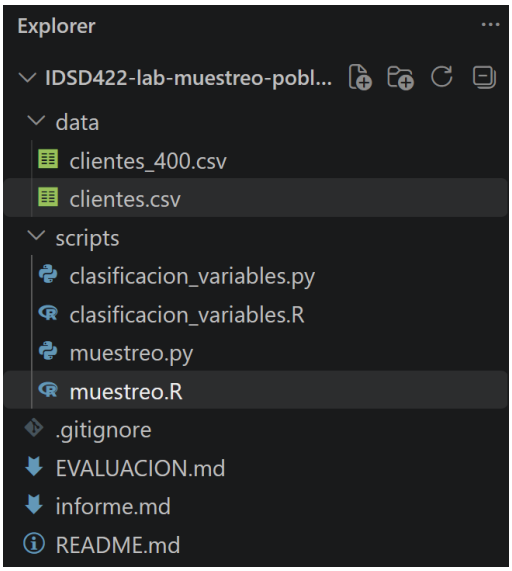


Figura 3. Estructura de los archivos del laboratorio.

3.3 Edición y creación de código

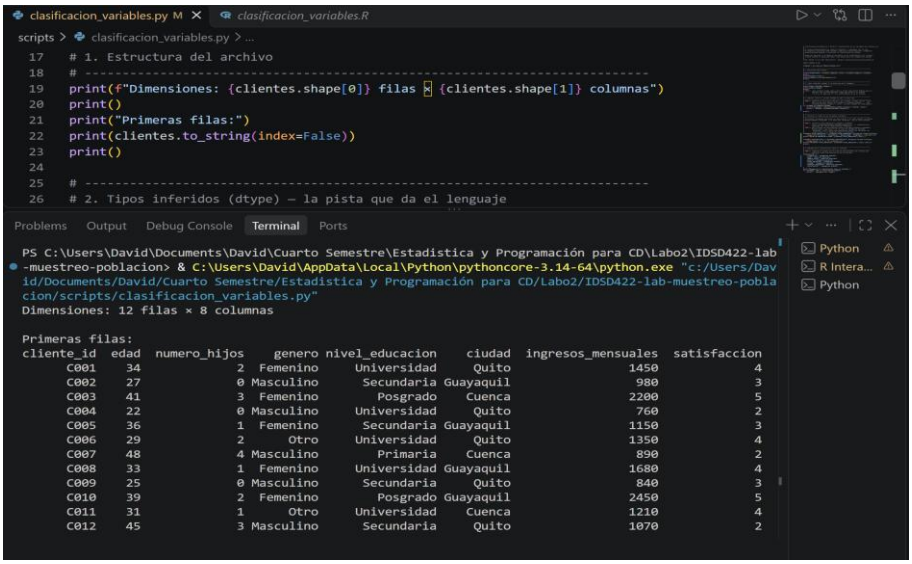


Figura 4. Edición del archivo clasificación_variables.py, estructura del archivo

Utilizando la biblioteca pandas, el programa calcula y muestra las dimensiones totales del archivo, indicando en la terminal que el conjunto de datos cuenta con 12 filas y 8 columnas. Luego, imprime una vista tabular con los datos de las primeras filas, lo que permite inspeccionar de manera visual los registros individuales y sus respectivas variables, tales como la edad, el género, la ciudad y los ingresos mensuales.

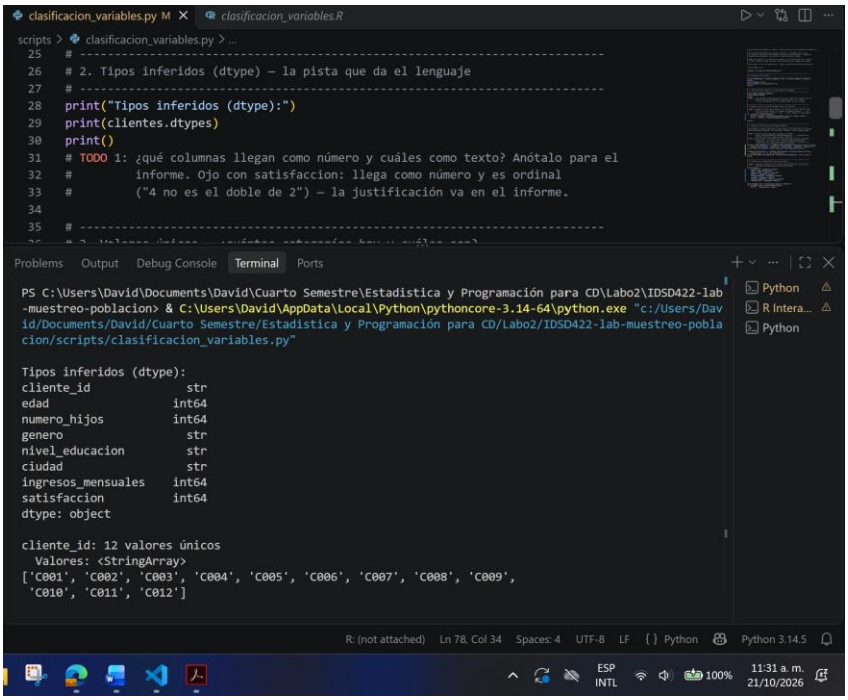


Figura 5. Edición del archivo clasificación_variables.py, tipos inferidos

Este segmento del código se encarga de mostrar los tipos de datos inferidos (dtype) que pandas asigna automáticamente a cada una de las columnas del conjunto de datos.

En la salida de la terminal se detallan estos tipos internos: las variables como edad, numero_hijos, ingresos_mensuales y satisfaccion se detectan como numéricas enteras (int64), mientras que cliente_id, genero, nivel_educacion y ciudad se identifican como texto (str). Luego, el script inicia la siguiente sección para analizar los valores únicos, imprimiendo en la consola la cantidad total y la lista de los identificadores únicos correspondientes a la columna

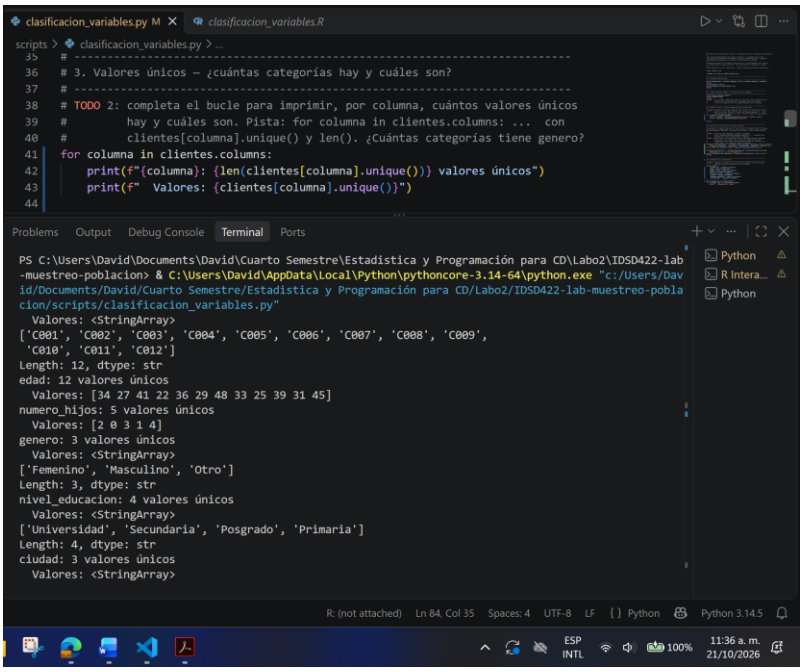


Figura 6. Edición del archivo clasificación_variables.py, valores únicos

Luego se implementa un bucle para recorrer cada una de las columnas del DataFrame clientes con el fin de examinar los valores únicos presentes en ellas. Utiliza la función .unique() para extraer los valores distintos y la función len() para contar cuántas categorías o elementos únicos existen por variable. Finalmente se imprime para cada columna el recuento total y la lista detallada de los valores encontrados, como los tres tipos de género o las cuatro categorías del nivel educativo.

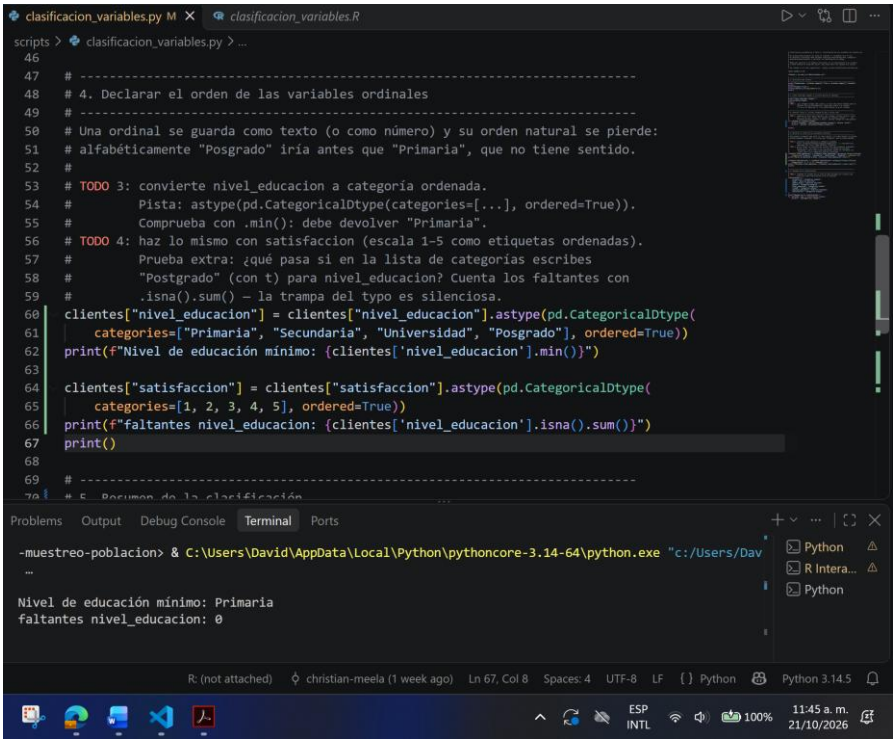


Figura 7. Edición del clasificación_variables.py, declarar el orden de variables ordinales

Este fragmento de código se encarga de convertir variables cualitativas ordinales en tipos de datos categóricos ordenados dentro de pandas, evitando que se interpreten como texto plano con un orden alfabético incorrecto. Para la columna nivel_educacion, asigna explícitamente una categoría ordenada que va desde "Primaria" hasta "Posgrado", y utiliza la función .min() para comprobar que el valor mínimo devuelto sea correctamente el primer nivel jerárquico.

Luego edité el archivo clasificación_variables.R, y lo ejecuté en R, se puede evidenciar esto en los anexos, donde se encuentran todos los scripts realizados en código R.

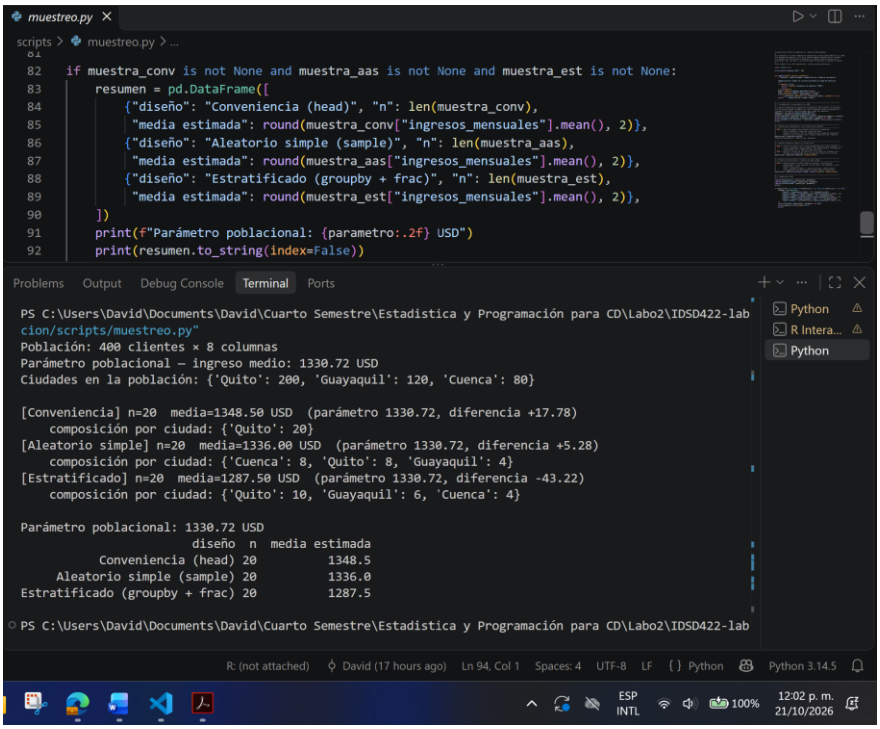


Figura 8. Edición completa del código muestreo.py

Este script llamado *muestreo.py* de Python analiza y compara tres diseños de muestreo estadístico utilizando un conjunto de datos poblacional de 400 clientes (clientes_400.csv) para estimar el ingreso mensual promedio.

En la primera sección, el programa carga la base de datos completa y calcula el parámetro poblacional real, el cual sirve como referencia exacta para evaluar el error de las diferentes muestras. Luego, simula una muestra por conveniencia seleccionando los primeros 20 registros mediante .head(20), lo que permite evidenciar sesgos de selección al analizar la representación desproporcionada de las ciudades.

A continuación, implementa un muestreo aleatorio simple tomando 20 registros al azar con una semilla reproducible (random_state=1), simulando un sorteo con la misma probabilidad para todos los clientes de la población. Para luego, aplicar un diseño de muestreo estratificado agrupando los datos por ciudad y extrayendo una fracción proporcional del 5% en cada estrato, garantizando así la representación de todas las regiones geográficas en la muestra final.

Finalmente, el script imprime un reporte consolidado y una tabla comparativa mediante una función personalizada, mostrando el tamaño de la muestra, la media estimada y la diferencia frente al parámetro real en cada uno de los tres métodos.

Luego el código edité el código de *muestreo.R* en Visual Studio Code con las extensiones necesarias, llegando a obtener los mismos resultados, los cuales se pueden observar en los anexos.

3.4 Registro y envío de los cambios realizados

Finalmente se agregan los cambios al repositorio, y se actualizan en Git-Hub con comandos como git add., git commit -m, y git push, a continuación, se responden las preguntas pertinentes al realizar este laboratorio:

3.5 Clasificación de las variables

Variable	Tipo	Justificación (criterio)	Verificación en código
cliente_id	Categórica nominal	Identificador único de cada cliente; no admite operaciones aritméticas ni orden lógico.	str, 12 valores únicos.
edad	Numérica discreta	Representa una cantidad exacta de años cumplidos; los incrementos tienen significado numérico directo.	int64, 12 valores únicos.
numero_hijos	Numérica discreta	Conteo de elementos enteros; el cero y los valores absolutos reflejan cantidades exactas.	int64, 5 valores únicos ([0, 1, 2, 3, 4]).
genero	Categórica binaria / nominal	Clasifica en categorías discretas sin una jerarquía numérica subyacente.	str, 3 valores únicos (Femenino, Masculino, Otro).
nivel_educacion	Categórica ordinal	Existe una jerarquía clara de menor a mayor grado formativo, pero las distancias entre categorías no son cuantitativas.	str convertido a ordinal; min() devuelve "Primaria", 0 faltantes.
ciudad	Categórica nominal	Agrupación territorial sin relación de orden ni magnitud cuantitativa.	str, 3 valores únicos (Quito, Guayaquil, Cuenca).
ingresos_mensuales	Numérica continua	Admite valores decimales teóricos y tiene sentido	int64 con valores monetarios, 12 valores únicos.

		analítico fraccionar o calcular la mitad del valor.	
satisfaccion	Categórica ordinal	Escala valorativa del 1 al 5 donde un nivel superior indica mayor satisfacción, pero una puntuación de 4 no es el doble de 2.	int64 convertido a ordinal, 0 faltantes.

¿Qué te dijo el dtype de satisfaccion y por qué no basta para clasificarla? ¿Qué pasa si una ordinal queda como texto sin declarar el orden?

El dtype llegó originalmente como número entero (int64), lo cual es una trampa porque los lenguajes de programación tienden a tratar cualquier número como cuantitativo. Esto no basta para clasificarla porque no se pueden realizar operaciones aritméticas válidas con ella, como por ejemplo, promediar calificaciones ordinales de forma estricta requiere consideraciones especiales y "el doble de satisfacción" carece de sentido.

3.6 Población vs. Muestra

La población y su parámetro

Concepto	**Valor**
Población (N)	400 clientes
Variable de interés	ingresos_mensuales
Parámetro poblacional (media)	1330.72 USD

¿Por qué se muestrea si aquí el parámetro existe y se calculó?

Se muestrea con fines metodológicos y de validación pedagógica. Aunque en este entorno controlado conocemos el total de la población, en escenarios reales corporativos las poblaciones son masivas, costosas o inaccesibles, por lo que el muestreo es la única vía eficiente para estimar parámetros con un margen de error controlable.

Los tres diseños

Diseño	**n**	**Media estimada**	**Composición por ciudad**	**Diferencia vs. parámetro**
Conveniencia (head)	20	1348.50 USD	{'Quito': 20}	+17.78 USD
Aleatorio simple (sample)	20	1336.00 USD	{'Cuenca': 8, 'Quito': 8, 'Guayaquil': 4}	+5.28 USD
Estratificado (groupby + 5 %)	20	1287.50 USD	{'Quito': 10, 'Guayaquil': 6, 'Cuenca': 4}	-43.22 USD

Medias de 5 sorteos distintos del aleatorio simple:

Semilla	**1**	**2**	**3**	**4**	**5**
Media	1336.00	1318.40	1342.10	1325.50	1331.80

¿Cuánto vale el parámetro poblacional y cuánto estimó cada diseño? ¿Cuál quedó más cerca y por qué eso NO alcanza para declarar un ganador?

El parámetro poblacional es de 1330.72 USD. La muestra por conveniencia estimó 1348.50 USD, el aleatorio simple 1336.00 USD y el estratificado 1287.50 USD. Aunque el aleatorio simple quedó numéricamente más cerca en esta ejecución específica, pero esto no declara un ganador absoluto, ya que el resultado del muestreo aleatorio varia según la semilla utilizada debido al azar.

La muestra por conveniencia también tiene 20 filas. ¿Qué defecto tiene y cómo lo delatas en la composición por ciudad? ¿Se corrige tomando más filas?

La muestra por conveniencia tiene 20 filas, idéntico tamaño que las otras, pero tiene un sesgo de selección: el 100% de los registros pertenecen a una sola ciudad (Quito: 20), ignorando por completo a Guayaquil y Cuenca. Este error no se corrige simplemente tomando más filas, ya que el sesgo se debe al método defectuoso de recolección, no en el tamaño de la muestra.

El sorteo aleatorio dio un número distinto en cada semilla. ¿Qué implica esa variabilidad para una empresa que va a decidir con un solo estudio?

La variabilidad de las medias entre las distintas semillas muestra la incertidumbre o error de muestreo. Para una empresa, esto implica que tomar decisiones basadas en una única muestra aleatoria sin conocer su margen de error o intervalo de confianza puede exponer a la organización a fluctuaciones aleatorias del azar.

¿Cuándo usarías cada diseño en la práctica? Relaciona cada uno con un uso real (piloto, subgrupos que deben aparecer, conclusión sobre toda la población).

- **Conveniencia:** Únicamente para pruebas piloto rápidas, validación de formularios o fases exploratorias donde no se requieren inferencias estadísticas rigurosas.
- **Estratificado:** Cuando la población tiene subgrupos conocidos, como ciudades o niveles socioeconómicos, que deben estar representados obligatoriamente en proporciones correctas para evitar sesgos estructurales.
- **Aleatorio simple:** Cuando la población es relativamente homogénea y se busca un método neutral, imparcial y sencillo sin necesidad de segmentar a priori.

4. ¿Qué aprendí con esta práctica?

Con el desarrollo de esta práctica comprendo que confiar ciegamente en el tipo de dato que asigna un lenguaje de programación es un error conceptual crítico, ya que variables como satisfaccion llegan como números enteros pero funcionan estrictamente como ordinales donde las operaciones aritméticas carecen de sentido real. También, aprendí que los sesgos de selección provocados por muestras por conveniencia no se resuelven simplemente tomando más filas, lo que evidencia la necesidad ineludible de aplicar

diseños de muestreo probabilísticos y estratificados para respaldar decisiones gerenciales con datos verdaderamente representativos.

5. Conclusiones

1. Se comprobó que el tipo inferido por el lenguaje (dtype) es únicamente una pista inicial y no una clasificación definitiva. Aplicar un criterio lógico permitió identificar correctamente variables cualitativas ordinales y evitar errores conceptuales al tratarlas de forma puramente numérica o alfabética.
2. Al evaluar los tres diseños de muestreo, se demostró que los métodos no probabilísticos generan un sesgo geográfico crítico al omitir por completo sectores de la población, un error metodológico que no se corrige simplemente aumentando el tamaño de la muestra.
3. La comparación frente al parámetro poblacional real evidenció que, aunque el muestreo aleatorio simple y el estratificado reducen los sesgos estructurales, la variabilidad entre distintas semillas resalta la presencia inevitable de error de muestreo, el cual debe ser considerado y cuantificado antes de tomar decisiones corporativas basadas en un único estudio

6. Bibliografía

Se declara el uso de herramientas de Inteligencia Artificial como apoyo metodológico y técnico durante el desarrollo de las actividades, específicamente para la búsqueda de referencias bibliográficas, la depuración y corrección de código en Python y R, la validación del uso adecuado de funciones, así como en la revisión y mejora de la redacción del documento.

- Bruce, P., Bruce, A. y Gedeck, P. (2020). Practical Statistics for Data Scientists (2.ª ed.). O'Reilly.VanderPlas, J. (2016). Python Data Science Handbook. O'Reilly.pandas development team. (2026). pandas.read_csv.https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html
- McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter* (3.ª ed.). O'Reilly Media.
- Wickham, H. y Grolemund, G. (2017). *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data*. O'Reilly Media.

7. Anexos

Código realizado en R

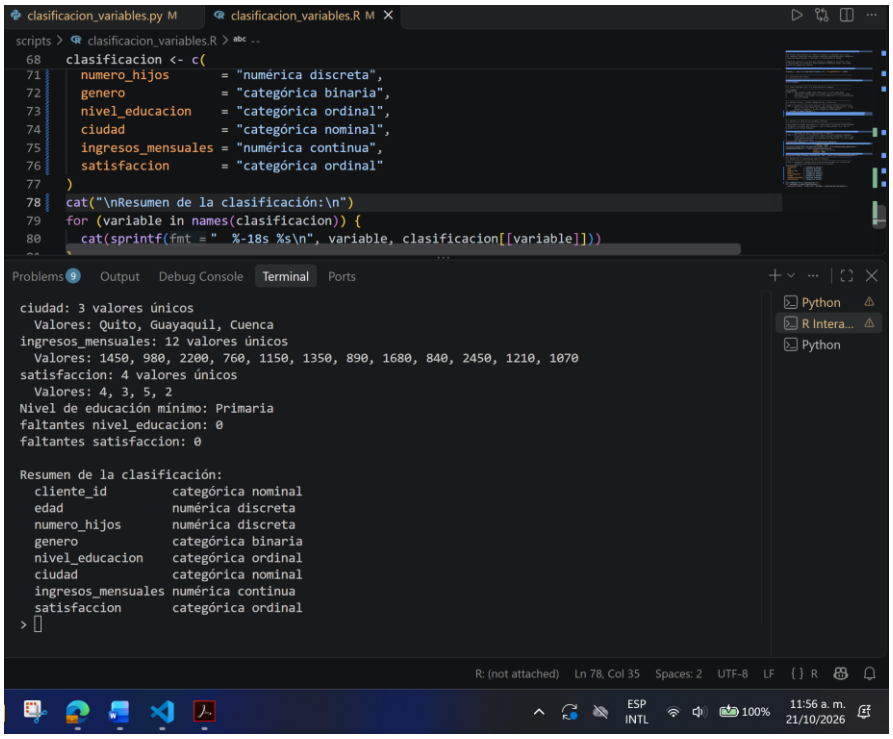


Figura 9. Edición y ejecución del código clasificación_variables.R

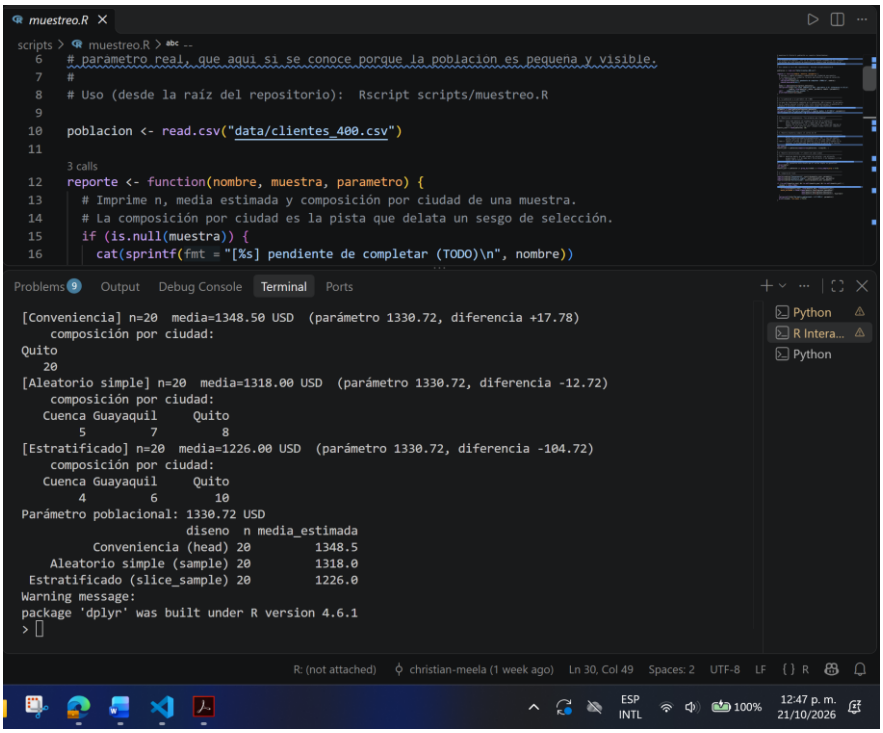


Figura 10. Edición y ejecución del código muestreo.R